10. 반복·연속 실행

지금까지 다룬 기능들은 모두 사용자가 에이전트에게 직접 지시하는 방식이었습니다. 하지만 "매일 아침 9시에 시스템 상태를 점검해줘"처럼 정해진 시각에 알아서 일하게 하거나, "이 프로젝트의 린트 에러를 전부 고쳐줘"처럼 끝날 때까지 스스로 턴을 이어가게 할 수도 있습니다. 이 장에서는 Hermes Agent의 두 가지 무인 실행 방식을 다룹니다.

- **크론**: 정해진 시각에 반복 실행합니다. 각 실행은 격리된 세션에서 독립적으로 처리됩니다.
- **영속 목표(`/goal`)**: 목표를 달성할 때까지 연속으로 턴을 이어갑니다. 매 턴 완료 후 판정 에이전트가 "다 됐는지"를 확인합니다.

이 장에서 다루는 내용

- 크론 스케줄링 (스케줄 형식, 작업 격리, SILENT 패턴, 보안)
- 실습: 자동 리포트 생성 (저장 구조, 조건부 알림)
- 운영 사례: 카페 모니터 크론 튜닝 (비용 진단, 모델 교체 dry-run, 하이브리드 재편)
- 영속 목표 — `/goal` 과 Ralph loop

10-1. 크론 스케줄링

> ⚠️ ***부분 검증**: v0.16.0 소스 코드 분석으로 확인했습니다. 게이트웨이 상주와 메시징 플랫폼 연동이 필요한 자동 틱 발화·외부 채널 전달 경로는 코드 분석 범위에 한정되며, 실제 발화 타이밍·전달 동작은 운영 환경에서 확인이 필요합니다.*

- 크론 스케줄링의 구조
- 스케줄 형식
- 크론 작업 등록
- 크론 실행 조건
- 정해진 시각에 어떻게 실행될까
- 작업 격리
- 프로젝트 디렉터리에서 실행: workdir
- 작업 체이닝: context_from
- [SILENT] 마커
- 결과 전달 옵션
  - 기본값
  - 홈 채널 지정
  - [SILENT]와의 관계
- 놓친 작업 처리
- 프롬프트 보안
- 크론 작업 관리
- 활용 사례
  - 시스템 운영
  - 개발 워크플로
  - 정보 수집
- 비용 고려사항
- 잡별 모델 지정

크론 스케줄링의 구조

Hermes Agent의 크론 기능은 대화 중에 자동화 작업을 등록하고, 정해진 시간에 에이전트가 자동으로 실행하는 기능입니다.

자연어 파서는 없습니다

크론 시스템에 별도의 NLP 자연어 파서가 내장되어 있는 것이 아닙니다. **LLM이 사용자의 자연어를 해석**하여 `parse_schedule()` 함수가 받아들이는 구조화된 형식으로 변환합니다. 즉, "매일 아침 9시"라고 말하면 LLM이 이를 `0 9 * * *`으로 변환하여 크론 작업을 등록합니다.

스케줄 형식

`parse_schedule()` 함수가 받아들이는 형식은 다음 네 가지입니다.

형식	예시	설명
기간 문자열	`30m`, `2h`, `1d`	지금부터 지정 시간 뒤 1회 실행
인터벌 문자열	`every 30m`, `every 2h`	지정 간격으로 반복
크론 표현식	`0 9 * * *`	표준 5필드 cron
ISO 타임스탬프	`2024-01-15T09:00:00`	특정 시각 1회 실행

사용자가 "매주 월요일 오전 10시"라고 말하면, LLM이 이를 `0 10 * * 1` 크론 표현식으로 변환하여 등록합니다.

크론 작업 등록

대화 중에 에이전트에게 자동화 작업을 요청합니다.

```
> 매일 아침 9시에 디스크 사용량을 확인해서 80% 이상이면 알려줘
```

에이전트가 `cronjob` 도구를 사용하여 크론 작업을 등록합니다.

다양한 스케줄 표현이 가능합니다.

```
> 매주 월요일 오전 10시에 Git 저장소의 미병합 브랜치를 정리해줘
> 매 시간마다 서버 응답 시간을 체크해줘
> 매월 1일에 지난 달 작업 통계를 정리해줘
> 평일 오후 6시에 오늘 커밋한 내용을 요약해줘
```

크론 실행 조건

크론 작업이 실행되려면 **게이트웨이가 실행 중**이어야 합니다.

```
hermes gateway    # 게이트웨이 실행 중이면 크론도 자동 실행
```

게이트웨이 없이 수동으로 크론 틱을 트리거하려면 다음 명령을 사용합니다.

```
hermes cron tick
```

정해진 시각에 어떻게 실행될까

이 책은 가급적 내부 구현을 깊이 다루지 않지만, 크론은 한 가지를 알아 두면 운영이 한결 편해집니다.

Hermes의 크론은 운영체제의 `crontab` 이나 `launchd` 를 쓰지 않습니다. 게이트웨이 프로세스 안에서 도는 백그라운드 스레드가 **60초마다 한 번씩** 등록된 작업 목록을 훑어 보고, 실행할 때가 된 작업을 즉시 실행합니다. 앞서 본 `hermes cron tick` 명령은 이 점검을 수동으로 한 번 더 돌리는 것으로, 게이트웨이가 자동으로 돌고 있을 때는 쓸 일이 없습니다.

여기서 두 가지 실용적인 결과가 따라옵니다.

- **해상도가 1분**: 크론 표현식의 분 단위까지가 한계입니다. "30초마다" 같은 스케줄은 지원하지 않습니다.
- **게이트웨이가 떠 있어야 작동**: 게이트웨이가 꺼져 있으면 누구도 시간을 보고 있지 않으므로 정해진 시각이 와도 작업이 돌지 않습니다. 9장에서 `hermes gateway install` (사용자 서비스로 등록), 리눅스 서버라면 `--system` (부팅 시 자동 기동)을 권장한 이유가 여기에 있습니다.

같은 점검이 동시에 두 번 도는 일도 막혀 있습니다. 게이트웨이의 점검 스레드와 수동으로 친 `hermes cron tick` 이 우연히 겹치더라도 한 번에 하나만 진행되므로, 같은 시각에 작업이 두 번 실행되는 일은 없습니다.

작업 격리

각 크론 작업은 **격리된 AIAgent 세션**에서 실행됩니다.

- 작업 간 컨텍스트가 공유되지 않습니다
- 이전 크론 실행의 메모리가 다음 실행에 전달되지 않습니다
- **반복 방지**: 크론 실행 중에는 `cronjob` 도구 세트가 비활성화됩니다. 크론 작업이 또 다른 크론 작업을 등록하는 재귀를 방지하는 안전장치입니다.

프로젝트 디렉터리에서 실행: `workdir`

기본적으로 크론 작업은 어떤 저장소와도 분리된 채 실행됩니다 — `AGENTS.md` · `CLAUDE.md` · `.cursorrules`가 로드되지 않고, 터미널·파일·코드 실행 도구의 작업 디렉터리는 게이트웨이가 시작된 위치 그대로입니다. 특정 프로젝트 폴더 안에서 작동하길 원하면 `workdir`을 지정합니다.

```
cronjob(
    action="create",
    schedule="매일 오전 9시",
    workdir="/home/me/projects/acme",
    prompt="이 프로젝트의 열린 PR을 점검해 #eng 채널로 요약을 보내줘",
)
```

`workdir`이 설정되면:

- 해당 디렉터리의 `AGENTS.md` · `CLAUDE.md` · `.cursorrules`가 시스템 프롬프트에 주입됩니다(인터랙티브 CLI와 같은 탐색 순서)
- `terminal`, `read_file`, `write_file`, `patch`, `search_files`, `execute_code` 모두 `TERMINAL_CWD`가 그 디렉터리로 설정된 채 실행됩니다
- **반드시 절대 경로**여야 하며, 존재하는 디렉터리여야 합니다 — 상대 경로나 없는 디렉터리는 등록·수정 시점에 거부됩니다
- 편집 시 빈 문자열(`workdir=""`)로 클리어하면 기본 작동으로 복귀합니다

`workdir`이 설정된 작업은 같은 틱에서 **순차로** 실행됩니다(병렬 풀에 들어가지 않음). `TERMINAL_CWD`가 프로세스 전역이라 두 workdir 작업이 동시에 돌면 서로의 cwd를 덮어쓰기 때문입니다. workdir이 없는 작업은 종전대로 병렬 실행됩니다.

작업 체이닝: `context_from`

다른 크론 작업의 **가장 최근 성공 출력**을 입력으로 받아오는 기능입니다. 작업 이름(또는 ID)을 `context_from`에 나열합니다.

```
cronjob(action="create", name="daily-digest",
        schedule="매일 오전 7시",
        context_from=["ai-news-fetch", "github-prs-fetch"],
        prompt="위 출력을 활용해 일일 다이제스트를 작성해줘.")
```

지정한 작업들의 최근 완료 출력이 프롬프트 위쪽에 컨텍스트로 주입됩니다. 주의: 같은 틱에서 같이 도는 상위 작업의 결과를 **기다리지 않습니다** — 그 시점에 저장된 가장 최근 출력만 읽습니다. 따라서 체인을 의도한다면 상위 작업이 먼저 끝나도록 스케줄을 어긋나게 잡아야 합니다.

[SILENT] 마커

크론 작업의 응답이 `[SILENT]`로 시작하면 결과 전달이 **억제**됩니다.

이 기능은 "이상이 있을 때만 알려줘" 패턴에 유용합니다. 프롬프트에 다음과 같이 지시하면 됩니다.

```
디스크 사용량을 확인하세요. 80% 이하이면 [SILENT]로 시작하는 응답을 작성하세요.
80%를 초과하면 경고 메시지를 작성하세요.
```

정상 상태에서는 알림이 오지 않고, 문제가 발생했을 때만 메시지를 받게 됩니다.

결과 전달 옵션

크론 작업의 결과를 어디로 보낼지 `deliver` 옵션으로 지정할 수 있습니다.

옵션	설명
`local`	로컬 파일에만 저장
`origin`	작업을 등록한 플랫폼으로 전달
`telegram`	Telegram 홈 채널로 전달
`discord`	Discord 홈 채널로 전달
`signal`	Signal로 전달
`sms`	SMS로 전달
`platform:chat_id`	특정 플랫폼의 특정 채팅으로 전달

기본값

`deliver`를 지정하지 않으면 다음 규칙으로 자동 결정됩니다.

- **텔레그램·디스코드 등 게이트웨이 채팅에서 잡을 등록한 경우** → `origin`. 잡을 등록할 때의 채팅 위치(플랫폼·채팅 ID)가 잡에 함께 저장되어 있다가 결과 회신 때 사용됩니다.
- **CLI 대화(`hermes`)에서 등록한 경우** → `local`. 외부 채널로는 알림이 가지 않고 로컬 파일에만 결과가 쌓입니다.

따라서 CLI에서 등록한 잡의 결과를 외부 채널로 받고 싶다면 명시해야 합니다. 자연어로 말해도 되며, 에이전트가 `cronjob` 도구 호출에 옵션을 끼워 넣습니다.

```
> 매일 아침 9시에 디스크 사용량 점검해서 텔레그램으로 보내줘
```

홈 채널 지정

`telegram` · `discord` · `signal` · `sms` 로 전달하려면 해당 플랫폼에서 미리 홈 채널을 지정해 둬야 합니다.

```
> /sethome
```

이 명령을 실행한 채널이 크론 결과를 받는 "홈 채널"이 됩니다. 9장에서 다룬 메시징 게이트웨이가 떠 있어야 함은 물론입니다.

메시징 게이트웨이와 결합하면, 아침에 출근하면 Telegram에 어젯밤 서버 상태 리포트가 와 있는 식의 운영이 가능해집니다.

`[SILENT]` 와의 관계

앞에서 본 `[SILENT]` 마커는 `deliver` 와 독립된 레이어입니다. `deliver` 가 무엇이든 응답이 `[SILENT]` 로 시작하면 전달이 건너뜁니다. "이상이 있을 때만 알려달라" 패턴은 `deliver="telegram"` (또는 `origin`)으로 받을 채널을 지정하고 프롬프트에 `[SILENT]` 분기를 둠으로써 만듭니다.

놓친 작업 처리

반복 작업의 예정 시간에 게이트웨이가 꺼져 있었다면, 다시 시작했을 때 놓친 작업을 **한꺼번에 몰아서 실행하지 않습니다**. 다만 짧게 놓친 경우에는 한 번만 따라잡기(catch-up) 실행을 합니다. 따라잡기를 허용하는 유예 구간은 스케줄 주기의 절반(최소 2분, 최대 2시간)이며, 이 구간을 넘기면 마지막 실행 시각을 갱신하고 다음 예정 시각부터 정상 실행됩니다(fast-forward).

예를 들어, 하루 1회 작업은 최대 2시간까지 늦더라도 재시작 시 1회 따라잡기 실행됩니다. 반면 매시간 작업이 3시간 동안 놓쳤다면 3번 몰아 실행하지 않고, 유예 구간(30분)을 넘긴 만큼 다음 정시에 1회만 실행합니다.

프롬프트 보안

크론 작업의 프롬프트는 등록 시 **보안 스캐닝**을 거칩니다.

- **인젝션 탐지**: 시스템 프롬프트 조작 시도 감지
- **데이터 유출 탐지**: 민감한 데이터를 외부로 전송하는 패턴 감지
- **비가시 유니코드**: 보이지 않는 유니코드 문자(zero-width space 등) 탐지

이 검사는 악의적인 프롬프트가 크론 작업으로 등록되는 것을 방지합니다.

크론 작업 관리

등록된 크론 작업을 확인하고 관리합니다.

```
> 등록된 크론 작업 목록을 보여줘
> 디스크 체크 크론을 삭제해줘
> 일일 리포트 크론의 실행 시간을 오전 8시로 변경해줘
> 모든 크론 작업을 일시 정지해줘
```

활용 사례

시스템 운영

- **일일 리포트**: 매일 아침 시스템 상태(CPU, 메모리, 디스크) 요약
- **야간 백업 확인**: 백업 스크립트 실행 후 성공 여부 확인 및 보고
- **로그 분석**: 매일 에러 로그를 분석하고 심각도별 분류

개발 워크플로

- **코드 품질**: 주기적으로 린트, 테스트 커버리지, 의존성 취약점 체크
- **브랜치 정리**: 매주 병합 완료된 브랜치를 목록으로 정리
- **의존성 업데이트**: 매주 업데이트 가능한 패키지 확인 및 보고

정보 수집

- **뉴스 브리핑**: 매일 특정 주제의 최신 정보 수집 및 요약
- **경쟁사 모니터링**: 정기적으로 특정 웹사이트 변화 체크
- **기술 트렌드**: 매주 GitHub 트렌딩 저장소 분석

비용 고려사항

크론 작업은 매 실행마다 LLM을 호출하므로, 비용을 고려해야 합니다.

- **로컬 LLM**: 전기 비용만 발생하므로 빈번한 스케줄도 부담이 적습니다
- **클라우드 API**: 실행 빈도와 작업 복잡도에 따라 토큰 비용이 발생합니다
- **권장**: 처음에는 일 1~2회 정도로 시작하고, 필요에 따라 늘려가는 것이 좋습니다

잡별 모델 지정

크론 잡은 메인 세션과 다른 모델·프로바이더를 쓰도록 설정할 수 있습니다. 평소 대화는 클라우드 프론티어 모델로 하면서, 매일 도는 단순 요약 잡은 로컬 모델로 돌리는 식의 분리가 가능합니다.

스케줄러는 잡 실행 시 다음 순서로 모델을 결정합니다.

1. 잡에 저장된 `model` · `provider` (있으면 우선)
2. 환경 변수 `HERMES_MODEL`
3. `config.yaml` 의 `model.default`

설정 경로는 두 가지입니다.

- **자연어 등록 시 함께 지정**: 잡을 등록할 때 모델·프로바이더를 함께 일러두면, 에이전트가 `cronjob` 도구 호출에 반영합니다.

  ```
  > 매일 아침 9시에 GitHub 트렌딩을 요약해줘. 모델은 로컬 qwen3:14b로 돌려.
  ```

- **`~/.hermes/cron/jobs.json` 직접 편집**: 잡 객체의 `"model"` · `"provider"` · `"base_url"` 키를 추가하거나 수정합니다.

`hermes cron create` 셸 명령에는 모델 지정 옵션이 아직 노출돼 있지 않습니다. CLI로 잡을 만든 다음 모델을 바꾸려면 `jobs.json` 을 직접 편집합니다.

프로바이더 고정 방식

자연어 등록에서 모델만 지정하고 프로바이더를 생략하면, 잡 생성 시점의 메인 프로바이더가 잡에 함께 저장됩니다. 이후 `hermes model` 로 메인 프로바이더를 바꿔도 잡은 옛 프로바이더로 계속 실행됩니다. 메인 프로바이더 변경에 자동으로 따르게 하려면 `jobs.json` 에서 해당 잡의 `provider` 필드를 `null` 로 비워 둡니다.

크론 작업 설계 팁

크론 작업의 프롬프트는 구체적으로 작성하세요. "시스템 상태를 확인해줘"보다 "디스크 사용량, 메모리 사용량, 상위 5개 프로세스를 확인하고 마크다운 테이블로 정리해줘"가 더 일관된 결과를 제공합니다. 이상이 없을 때 알림을 받고 싶지 않다면 `[SILENT]` 패턴을 활용하세요.

10-2. 실습: 자동 리포트 생성

> ⚠️ ***부분 검증**: 크론 저장 구조와 동작(저장 경로·동시 실행 방지·게이트웨이 의존·수동 `tick`)은 v0.16.0 소스 코드와 대조해 확인했습니다. 다만 다섯 가지 실습의 실제 등록·실행 흐름은 실행 테스트로 검증하지 않았습니다.*

- 크론 저장 구조
- 실습 1: 간단한 크론 작업
- 실습 2: 일일 시스템 리포트
- 실습 3: 조건부 알림 (SILENT 패턴)
- 실습 4: 크론 + 메시징 연동
- 실습 5: 크론 작업 관리
- 주의사항

이 절에서는 크론 기능의 기본 사용법, 자동 시스템 리포트 생성, 조건부 알림, 메시징 연동, 작업 관리까지 다섯 가지 실습을 차례로 진행합니다.

크론 저장 구조

크론 작업의 데이터는 다음 위치에 저장됩니다.

```
~/.hermes/cron/
├── jobs.json                    # 모든 크론 작업 정의
└── output/
    └── {job_id}/
        └── {timestamp}.md       # 각 실행 결과
```

- `jobs.json` : 모든 크론 작업이 하나의 JSON 파일에 저장됩니다. 스케줄, 프롬프트, 활성화 상태, 마지막 실행 시각 등이 포함됩니다.
- `output/{job_id}/{timestamp}.md` : 각 실행 결과가 작업 ID별 디렉터리에, 실행 시각을 파일명으로 하여 마크다운 파일로 저장됩니다.

동시 실행 방지

크론 스케줄러는 실행 중인 작업 ID를 추적해 동일 작업의 동시 실행을 방지합니다. 이전 실행이 아직 끝나지 않았다면 다음 틱에서 그 작업은 건너뜁니다. 오래 걸리는 작업이라도 중복 실행 걱정 없이 안전합니다. 이와 별개로, 게이트웨이의 자

동 틱과 수동 `hermes cron tick`이 겹쳐도 한 번에 하나의 틱만 돌도록 파일 락(`~/.hermes/cron/.tick.lock`)이 쓰입니다.

실습 1: 간단한 크론 작업

```
> 30분마다 현재 시각과 시스템 업타임을 /tmp/status.log에 기록하는 크론을 만들어줘
```

등록 후 확인:

```
> 등록된 크론 작업 목록을 보여줘
```

등록된 작업은 `~/.hermes/cron/jobs.json`에서 직접 확인할 수도 있습니다.

```
cat ~/.hermes/cron/jobs.json | python3 -m json.tool
```

실습 2: 일일 시스템 리포트

```
> 매일 오전 9시에 다음 내용을 포함하는 시스템 리포트를 ~/reports/ 디렉터리에 생성해줘:
> - 디스크 사용량
> - 가장 큰 파일 TOP 10
> - 최근 24시간 내 수정된 파일
> 파일명은 report-YYYY-MM-DD.md 형식으로
```

리포트 결과는 두 곳에 저장됩니다.

1. 에이전트가 생성한 `~/reports/report-YYYY-MM-DD.md` 파일
2. 크론 시스템의 실행 기록: `~/.hermes/cron/output/{job_id}/{timestamp}.md`

실습 3: 조건부 알림 (SILENT 패턴)

이상이 있을 때만 알림을 받는 크론 작업입니다.

```
> 매 시간마다 디스크 사용량을 확인해줘. 80% 이하이면 [SILENT]로 시작하는 응답을 작성하고,
> 80%를 초과하면 경고 메시지를 보내줘.
```

정상 상태에서는 알림이 오지 않고, 문제가 발생했을 때만 메시지를 받게 됩니다.

실습 4: 크론 + 메시징 연동

메시징 게이트웨이가 실행 중이라면, 크론 결과를 메시징으로 받을 수 있습니다.

1. Telegram이나 Discord에서 `/sethome` 실행
2. 크론 작업 결과가 해당 채널로 전송됨

실습 5: 크론 작업 관리

```
> 등록된 크론 작업 중 status.log 관련 작업을 삭제해줘
> 일일 리포트의 실행 시간을 오전 8시로 변경해줘
```

주의사항

- **게이트웨이 필수**: 크론 작업이 자동 실행되려면 `hermes gateway`가 실행 중이어야 합니다. 게이트웨이 없이는 `hermes cron tick`으로 수동 실행만 가능합니다.
- **비용**: 크론 작업은 LLM을 호출하므로, 빈번한 스케줄은 토큰(또는 컴퓨팅) 비용이 발생합니다.
- **로컬 LLM**: 비용 부담은 없지만, 동시 요청 시 응답 속도가 느려질 수 있습니다.
- **에러 처리**: 중요한 작업의 크론은 에러 처리를 포함하도록 지시하세요.
- **격리**: 각 크론 작업은 독립된 세션에서 실행됩니다. 이전 실행 결과를 참조하려면 파일로 저장하도록 지시해야 합니다.
- **재귀 방지**: 크론 실행 중에는 새로운 크론 작업을 등록할 수 없습니다.

10-3. 운영 사례: 카페 모니터 크론 튜닝

> ℹ️ ***실측 사례**: 2026-05-17에 실제 운영 중인 cron을 대상으로 진행한 진단·dry-run·재편 기록입니다. 분석·재편 작업의 메인 모델은 `claude-opus-4.7` (OpenRouter)을 썼습니다. 단순 등록과 달리 다단계 추론·아키텍처 설계가 필요해서 고른 모델입니다. 잡 자체가 도는 모델은 별개로 두는 게 정상이며, 이 사례에서도 잡은 Kimi K2.6에서 시작해 재편 결과로 다른 모델로 갈아탑니다.*

- 배경 — 스터디 모니터링을 Hermes로 옮긴 흐름
- 운영 중인 잡 — naver-cafe-monitor
- 첫 자연어 질문
- AM/PM 비교 — 같은 잡인데 비용이 두 배
- 첫 권고 — 시행착오를 영구화하라
- 모델 교체 dry-run
- 네 가지 옵션과 추천
- "b" 한 글자에서 완주까지
  - 구현하면서 마주친 함정 셋
  - End-to-end 결과
- 메모리가 한 일
- 비용 최적화의 역설
- 정리 — 이 사례에서 얻은 것

이 절에서는 며칠 동안 운영한 cron의 비용 패턴이 이상해 보여 들여다본 과정을 그대로 따라갑니다. 10-1·10-2가 잡을 "등록"하는 그림이라면, 여기서는 **이미 도는 잡을 자연어로 진단하고 재편**합니다. 사용자가 어디부터 봐야 할지 미리 짚어 주지 않아도 된다는 점이 핵심입니다.

배경 — 스터디 모니터링을 Hermes로 옮긴 흐름

한 출판사가 운영하는 네이버 카페에서 독자 대상 스터디 그룹을 운영하고 있고, 저는 참가자들이 올리는 글과 댓글을 하나하나 읽으며 누가 어디서 막혔는지, 혼자 끙끙대는 질문이 없는지 살피고 있습니다. 그동안은 전통적인 크롤링과 시스템 `crontab`으로 일일 모니터링을 돌려 왔습니다. 글을 긁어 와서 필터링하고 결과를 받아 보는 식이었습니다.

Hermes로 옮기려 한 이유는 단순 자동화가 아니었습니다. 핵심은 **지능적 선별**이었습니다. 기본 요건은 한 줄로 정해졌습니다. "참가자가 스터디에 문제를 겪는 경우 macOS 알림 배지를 띄우고, 클릭하면 곧장 해당 카페 게시물 URL로 이동할 것."

첫 자연어 등록은 단번에 끝나지 않았습니다. 처음에는 크론 하나가 크롤링·이슈 선별·알림 발송을 모두 맡았습니다. 막상 돌려 보니 문제가 있었습니다. "어떤 글을 이슈로 봤는지" 기록이 어디에도 남지 않아 같은 글을 실행할 때마다 새로 분류하고 있었습니다. Linear를 이슈 트래커로 도입해 구조를 가르고(이슈 발견과 처리 상태를 Linear에 외부 저장), 하루 정도 돌린 시점에 OpenRouter 비용을 들여다보게 됐습니다. 이 절은 **그 시점부터**의 기록입니다.

운영 중인 잡 — `naver-cafe-monitor`

매일 3회(09:30, 13:30, 17:30) 도는 잡입니다. Selenium으로 카페 글을 수집하고, Linear에 이슈로 동기화하고, 변화가 있으면 Apple Reminders로 알림을 등록(클릭하면 해당 카페 게시물 URL로 이동)하고, 실행 로그를 Memos에 기록하는 22~37턴짜리 워크플로입니다. 며칠 돌리고 보니 한 번 실행에 OpenRouter 비용이 약 \$0.5~\$1.0씩 잡혔습니다.

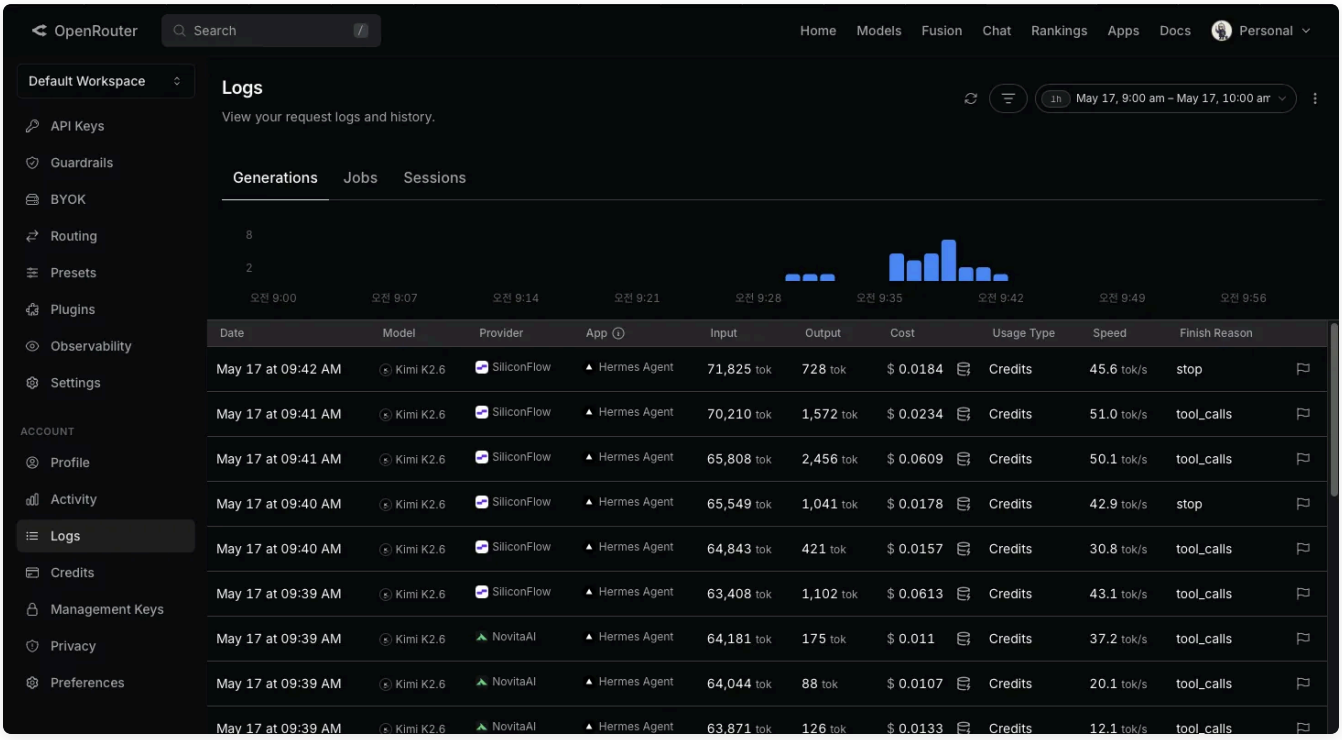
첫 자연어 질문

사용자가 던진 첫 질문은 평이했습니다.

```
> 지난 24시간 호출 패턴 보고 어디서 토큰을 많이 쓰는지 짚어줘.
```

"OpenRouter 로그 어디 봐줘"도 아니고 "세션 JSONL 파싱해줘"도 아닙니다. 그냥 의도만 던졌습니다. 에이전트는 잡 정의·세션 기록·OpenRouter 로그가 어디에 있는지 알기 때문에, 사용자가 어디부터 봐야 할지 짚어 주지 않아도 됩니다.

OpenRouter 대시보드의 같은 시간대 로그는 다음과 같습니다.



Date (5/17)	Model	Input	Output	Cost	Finish Reason
09:42 AM	Kimi K2.6	71,825 tok	728 tok	\$0.0184	stop
09:41 AM	Kimi K2.6	70,210 tok	1,572 tok	\$0.0234	tool_calls
09:41 AM	Kimi K2.6	65,808 tok	2,456 tok	\$0.0609	tool_calls
09:40 AM	Kimi K2.6	65,549 tok	1,041 tok	\$0.0178	stop
09:40 AM	Kimi K2.6	64,843 tok	421 tok	\$0.0157	tool_calls
…	…	…	…	…	…

입력 토큰이 한 호출에 65K~71K이고, 같은 잡 한 번에 호출이 11~13건씩 쌓이는 모양입니다.

AM/PM 비교 — 같은 잡인데 비용이 두 배

같은 잡인데 AM(09:30) 세션과 PM(17:30) 세션의 호출 수·총 비용이 달랐습니다.

세션	어시스턴트 턴	OpenRouter 호출	총 토큰	총 비용
AM 09:30	22	25	0.9M	\$0.49
PM 17:30	22	37	2.0M	\$1.02

세션 안 어시스턴트 턴은 22로 같은데 OpenRouter 호출은 +12건 더 있었고, 비용은 약 2.1배입니다.

에이전트는 두 세션의 JSONL 파일을 직접 파싱해 단계별로 매핑했습니다. 결과는 다음과 같았습니다.

단계 묶음	AM	PM
부트스트랩 (env·config·script 확인)	1 턴	3 턴
데이터 수집 (Selenium·6분)	1 턴	1 턴
`cafe_data.json` 분할 읽기	2 턴	2 턴
Linear API 호출	2 턴 (issueCreate 일발 성공)	**12 턴 (mutation 디버깅 6라운드)**
Apple Reminders 등록	2 턴	2 턴
Memos 전송	4 턴 (MEMOS_USER 미설정으로 실패)	2 턴 (성공)
최종 보고	1 턴	1 턴

PM 폭증의 직접 원인은 분명했습니다. **Linear GraphQL mutation의 스키마를 매번 새로 시행착오로 발견**하느라 디버깅 6라운드(write_file → terminal 12연속)가 들어갔습니다. AM에는 같은 mutation이 일발에 통과했고, 그래서 12턴이 비었습니다.

첫 권고 — 시행착오를 영구화하라

에이전트가 이 진단을 마치고 곧장 권고를 줬습니다.

- PM의 `update_summary_fixed.py`를 영구 경로로 옮겨 매 실행마다 재발견하지 않도록 고정. 다음 PM 실행은 호출 수가 AM 쪽(~25건)으로 수렴할 것입니다.
- Memos 전송이 AM에는 실패하고 PM에는 성공한 이유는 PM 부트스트랩에 `set -a; source ~/.hermes/.env; set +a`가 들어가 있었기 때문입니다. 같은 패턴을 AM 프롬프트에도 통일.

여기까지가 "비용 패턴 한번 봐 줘"라는 한 줄에서 출발한 흐름입니다.

모델 교체 dry-run

비용을 더 줄여 보려고 사용자가 `~/.hermes/cron/jobs.json`을 직접 손봐 모델을 로컬 `gemma4:26b`(데스크톱 RTX 4090의 Ollama 서버 `http://<Ollama-서버-IP>:11434/v1`)로 바꿨습니다. 그러고 한마디 했습니다. "수작업으로 크론 설정을 로컬 모델로 바꿨다. 지금 한번 테스트해 볼까?"

에이전트는 먼저 서버가 살아 있는지 확인했습니다. 그다음 `calculator` 도구 하나만 노출해 간단한 시험 콜을 날렸고, 마지막으로 실제 크론 프롬프트(6,151자)와 도구 셋을 붙여 dry-run을 돌렸습니다. 앞 두 단계는 정상이었는데 마지막에서 막혔습니다. 첫 턴 응답 시간 120초, `finish_reason=stop`, `tool_calls=0`. 도구를 하나도 부르지 않고 "명세를 이해했습니다"

같은 메타 코멘트만 돌려보냈습니다. Hermes의 `clarify` 단계가 120초 timeout으로 끝나자 에이전트는 자체 판단으로 세 모델 순차 dry-run을 백그라운드로 띄웠습니다.

모델	시간	tool_calls	평가
`qwen3:14b`	300s+	(timeout)	응답 자체 없음. 로딩 + reasoning 토큰 폭증 의심
`hermes3:latest`	187s	0	Python 코드 스니펫만 출력
`gemma4:26b`	59s	1 (todo)	도구는 부르지만 스키마 어긋남

종합 진단: 22~37턴 다단계 워크플로에 14B/26B 로컬 모델은 아직 부적합. Linear mutation 디버깅 구간에서 무한루프에 빠질 위험이 큽니다.

로컬 모델로 갈 때 막상 부딪히는 벽

한국어 보고 품질만 보면 Gemma 4 26B가 RTX 4090에서 쓸 만합니다. 단발 분류·요약에는 충분합니다. 다만 에이전틱 워크플로(여러 턴에 걸쳐 일관된 도구 호출)에서는 instruct 튜닝의 방향이 결과를 갈라 놓습니다. 도구 호출은 단순 시험 콜로는 검증되지 않으니, **실제 프롬프트로 dry-run**을 한 번 거치는 게 안전합니다.

네 가지 옵션과 추천

결국 로컬 모델로 그냥 갈아 끼우는 길은 막혔습니다. LLM 추론이 필요 없는 구간을 먼저 걷어내야 다음 시도도 할 수 있습니다. 로컬 모델이든 클라우드 모델이든 마찬가지입니다. 이 판단을 바탕으로 에이전트는 네 가지 선택지를 제시했습니다.

- **(a) 클라우드 모델 변경** — Kimi K2.6 → `qwen3.6-plus` (메모리에 기록된 사용자 선호). 비용은 줄고 한국어 분석 품질은 개선될 가능성
- **(b) 하이브리드** — 워크플로를 4계층(수집·분류·동기화·알림)으로 가르고, 수집·동기화·알림은 규칙 기반 Python 스크립트로 분리해 `no_agent=True` cron으로 돌리며, LLM이 정말 필요한 분류 단계만 단발 API 콜로 빼냄
- **(c) 로컬 + 프롬프트 대수술** — 도구 스키마를 더 엄격히 명세하고 첫 턴 도구 호출을 few-shot 예시로 넣는 시도. 시간 투자 대비 회수가 불확실
- **(d) 롤백** — Kimi K2.6으로 되돌리고 신중히 결정

에이전트의 추천은 **(b) 하이브리드**였습니다. 이유는 명확합니다. PM 세션에서 본 12턴짜리 디버깅 루프는 LLM이 매번 다시 할 일이 아니었습니다. 한 번 스크립트로 고정해 두면 끝나는 작업이었습니다. 비용의 대부분이 "스크립트 재발견·디버깅"에 쓰였다면, 그 부분은 LLM 바깥으로 빼는 게 정답입니다.

"b" 한 글자에서 완주까지

사용자는 `b` 한 글자로만 답했습니다. 에이전트가 펼친 4계층은 다음과 같습니다.

```
Layer 1 (수집)   — Selenium 카페 크롤링 (규칙 기반)
Layer 2 (분류)   — 운영진 댓글 감지 + LLM 단발 콜 (필요할 때만)
Layer 3 (동기화) — Linear 이슈 자동해결·신규생성·요약 갱신 (규칙 기반)
Layer 4 (알림)   — 변화 감지 시 Apple Reminders + Memos 기록 (규칙 기반)
```

오케스트레이터 `~/.naver_cafe/run.py`가 네 계층을 순서대로 호출하고, cron은 `no_agent=true script=...`로 등록만 합니다. LLM 호출은 Layer 2의 카테고리 분류 폴백뿐(보통 0~1회/실행).

설계 후 에이전트는 PM 세션이 `/tmp`에 흩어 놓은 옛 스크립트들을 인벤토리로 정리하고, 거기서 코드 문제점(`headless=False`로 cron에서 GUI가 뜸, `TODAY` 하드코딩, `osascript -e` 인라인 사용)을 자기가 찾아냈습니다. 이렇게 하면 예전 시행착오의 결과물이 "이번 한 번 쓰고 버릴 코드"로 끝나지 않습니다. 다음 실행부터 계속 재사용할 자산이 됩니다.

구현하면서 마주친 함정 셋

한 계층씩 만들고 검증하는 사이에 세 가지 함정이 새로 드러났고, 그때마다 에이전트가 진단하고 해결한 뒤 **스킬 문서에 영구 저장**했습니다.

함정	해결	위치
Python.org Python 3.11은 CA 번들이 비어 `urllib`가 `SSLCertVerificationError`로 실패	`"/Applications/Python 3.11/Install Certificates.command"` 1회 실행	환경
백그라운드 프로세스의 stdout이 Hermes `process(action='poll')`에 실시간으로 안 잡힘 → 정상 진행 중인 collect.py를 4분 30초 멎은 줄 알고 죽일 뻔함	모든 layer에 `/tmp/cafe_*.log` 듀얼 쓰기 추가, `tail -f`로 추적	Hermes
로컬 `gemma4:26b`가 `{"category":"개발환경"}` 같은 짧은 JSON에 대해 `max_tokens=50/200`이면 `finish_reason=length`로 빈 응답 반환	`max_tokens=1000`으로 늘리고 `finish_reason`을 로그에 같이 기록	LLM

다음에 같은 작업을 하는 누구든(에이전트 자신도 포함) 같은 자리에서 멈추지 않습니다. 메모리·스킬 시스템(5장·7장)의 실제 효용을 이 구간에서 확인할 수 있습니다.

End-to-end 결과

전체 파이프라인을 한 번 돌려 보고 검증한 결과:

계층	소요	비고
collect	288초	Selenium, 인증 25개 + 질문 4개
classify	16초	LLM 1회 호출 (구독·결제 → 개발환경 정확 분류)
sync_linear	0.5초	일일 요약 이슈 갱신, signature 일치 시 skip
notify	1.0초	Reminder 중복 안 만듦
합계	**305초**	두 번째 실행에서 idempotent 확인

cron은 `no_agent=true` + 4계층 스크립트 호출 wrapper로 변환됐고, **회당 LLM 비용이 \$0.5~\$1.0에서 사실상 \$0으로 떨어졌습니다.** Layer 2의 단발 분류 콜이 로컬 모델에서 처리되어 외부 과금이 없습니다.

> 💡 *흥미로운 부산물: 규칙 기반 분류기가 17:30 cron의 K2.6 에이전트보다 더 정확했습니다. K2.6이 놓친 `follow_up_needed` 케이스 두 건을 substring 매칭 기반 분류기가 깔끔하게 잡았습니다. 모든 의미 판단을 LLM에 맡기는 게 늘 우월하지 않다는 사례입니다.*

메모리가 한 일

이 흐름에서 두 군데 메모리가 큰 역할을 했습니다.

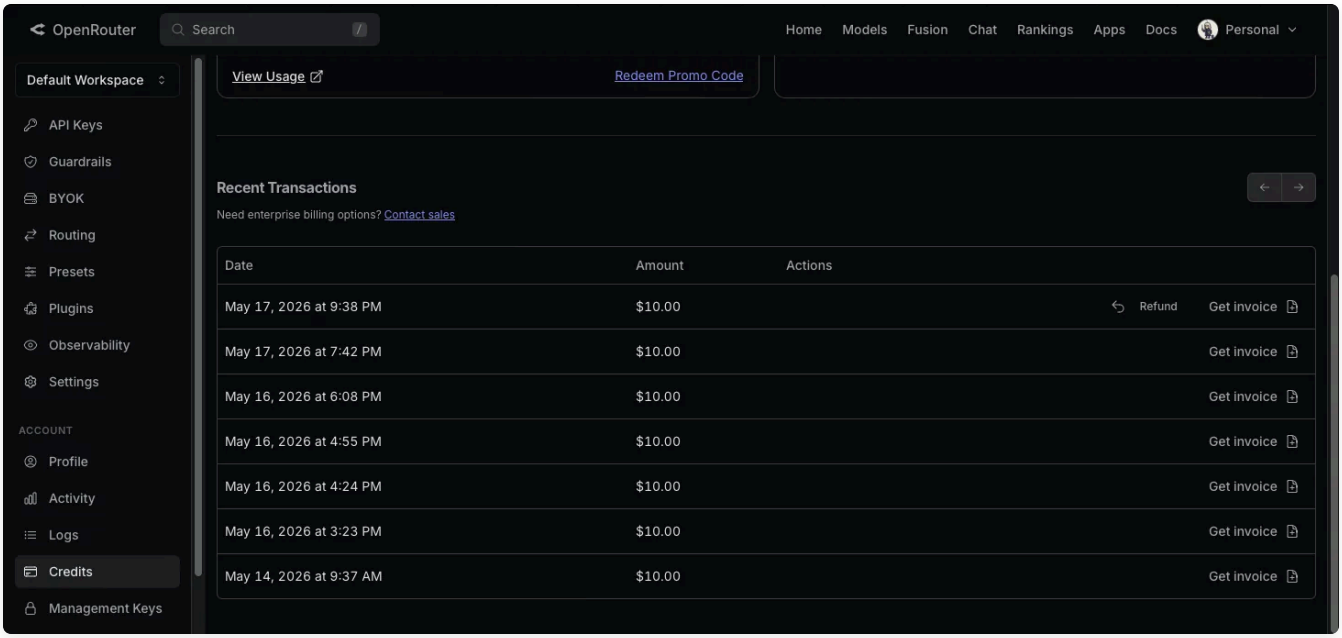
- `qwen3.6-plus가 카페 분석에 K2.6보다 우수` : 옵션 (a)에서 에이전트가 막연히 "더 싼 클라우드 모델" 같은 일반론을 댄 게 아니라 특정 모델을 짚어 추천했습니다.
- `SoC·직교축 분리를 중요시함 / automation 스크립트는 각 단계 수동 테스트 후 합치는 방식 선호` : 4계층 아키텍처를 굳이 설계해 보여 준 이유, 그리고 한 계층씩 검증해 가며 진행한 이유가 모두 이 메모리에서 나왔습니다.

이 사례는 메모리가 단순한 "사실 저장소"가 아니라 같은 자연어 지시에도 응답 방식을 바꾼다는 사실을 잘 보여 줍니다(5장 참고).

비용 최적화의 역설

이 케이스 스터디에는 솔직히 적어 둘 한 가지가 있습니다. **이 글에서 다루는 분석·재편 작업은 오늘 저녁 한 세션으로 진행됐고, 그 한 세션의 분석을 도와 준 메인 모델(`claude-opus-4.7`)에만 약 \$20~\$30(몇 만 원) 어치 크레딧이 들어갔습니다.** 잡 자체의 한 번 실행 비용이 \$0.5~\$1.0(약 몇 천 원) 수준이었던 것과 비교하면, "잡 비용 줄여 보자"는 분석 한 세션이 잡의 한 달치 운영 비용에 맞먹는 셈입니다.

시야를 넓혀 잡을 처음 만들기 시작한 시점부터 보면, 5월 16일(시스템 구축·Linear 도입·아키텍처 발전·알림 정책 재설계)에 \$10짜리 충전이 네 번, 5월 17일(본 케이스 스터디의 분석·재편 세션)에 두 번, 합쳐 이틀에 \$60 어치 충전이 일어났습니다. 그 외 5월 14일의 \$10 한 건은 잡과 무관한 별개 작업입니다.



잡 한 번 실행에 \$0.5~\$1.0이 든다는 점과 비교하면, **운영 사이클 전체로 보든 한 세션의 분석만 보든 잡 비용의 수십 배가 들어간** 셈입니다.

여기서 얻을 교훈은 두 가지입니다.

- **분석·재편을 시작하기 전에 ROI를 가늠해 볼 것**. 잡 한 번 \$0.5이고 하루 3회 도는 잡이면 한 달 \$45 수준입니다. 강한 모델로 며칠에 걸쳐 분석한다면 그 비용이 잡 자체의 몇 달 치를 넘어설 수 있습니다. 본격 재편이 정말 필요한 시점인지, 단순히 모델 한 줄 갈아 끼우는 정도로 충분한지 먼저 판단할 가치가 있습니다.
- **분석 모델 선택은 작업 무게에 맞춥니다**. 강한 모델을 골랐다는 사실 자체가 후회 거리는 아닙니다. 잡을 처음 만들 때(5월 15~16일)는 호출당 비용이 더 낮은 `qwen/qwen3.6-plus` (1M 컨텍스트·thinking 모드 토글을 갖춘 모델)로 시험해 봤고, 결과물이 그리 나쁘지 않다는 평가를 메모리에 남겨 둔 것도 그 시점입니다. 오늘(5월 17일) 작업은 분석·재편·아키텍처 설계가 한 세션 안에 누적되는 무게여서, 깊은 추론과 능동성이 긴 호흡으로 끝까지 유지될 모델이 필요하다고 보고 의식적으로 `claude-opus-4.7`을 골랐습니다. 두 모델 모두 다단계 추론 자체는 다루지만, 작업의 길이와 누적 부담을 안정적으로 감당하는 쪽으로 한 단계 올린 셈입니다. 이 사례에 드러난 능동성·구조 설계·환경 진단의 깊이가 그 선택의 결과입니다. 즉 모델은 작업 성격에 맞춰 그때그때 다르게 고르는 게 자연스럽고, 본 사례의 \$20는 그 선택의 명시적 가격표입니다.

즉 에이전트의 능동성과, **그 능동성이 곧 비용으로 직결된다는 면**이 한 사례 안에 같이 들어 있습니다.

정리 — 이 사례에서 얻은 것

- **자연어 인터페이스는 등록 이후에도 계속 쓰입니다**. 며칠 운영한 뒤의 진단·모델 교체·아키텍처 재편까지 같은 대화창에서 처리합니다. 사용자가 어디부터 봐야 할지 미리 짚어 줄 필요가 없습니다.

- **큰 방향만 말해도 에이전트가 다음 일을 찾아갑니다**. 사용자는 `b` 한 글자만 입력했습니다. 그 뒤 에이전트가 4계층 설계, 기존 스크립트 인벤토리, 환경 진단, 구현, 검증까지 이어서 처리했습니다. 모델을 바꾸자고 했을 때도 단순히 "바꿨습니다"가 아니라 서버 가용성·tool calling·실제 프롬프트 dry-run을 자기가 설계해 실행한 뒤 결과를 보고합니다.
- **결정이 필요한 순간에는 멈춥니다**. `clarify` timeout이 와도 마음대로 진행하지 않습니다. 잡이 깨질 수 있는 선택지는 먼저 위험을 알리고 사용자의 결정을 기다립니다. 이 지점이 안전판 역할을 합니다.
- **발견한 함정은 스킬에 그대로 적힙니다**. 이 사례에서 만난 세 함정(Python 3.11 SSL, Hermes stdout 캡처, gemma `max_tokens`)은 모두 그 자리에서 스킬 문서에 추가됐습니다. 다음에 같은 길을 갈 누구든 같은 자리에서 멈추지 않습니다. 메모리·스킬 시스템(5장·7장)의 실제 효용이 여기에 있습니다.
- **분석에는 강한 모델을 쓰고, 실제 잡은 가볍게 돌립니다**. 이 사례는 분석을 `claude-opus-4.7`로 돌렸고, 결과로 잡은 하이브리드(규칙 기반 처리 + 단발 로컬 LLM 콜)로 갈아탔습니다. 회당 비용이 \$0.5~\$1.0에서 사실상 \$0으로 떨어졌습니다.
- **로컬 모델로 안 되는 일도 분명히 적어 둡니다**. RTX 4090에서 14B/26B 모델은 단발 분류엔 충분하지만, 22턴 이상의 다단계 도구 호출 워크플로엔 2026년 5월 현재 아직 부적합합니다. 부록의 「로컬 LLM 가이드」와 함께 보면 그림이 맞아 들어옵니다.

앞선 장들이 Hermes의 기능을 절 단위로 다뤘다면, 이 절은 그 기능을 **며칠 굴려 보고 무엇이 부족했는지, 어떻게 다시 짜야 했는지**를 그대로 옮긴 기록입니다. 같은 흐름을 자기 cron에도 적용해 보시기 바랍니다.

10-4. 영속 목표

> *v0.14.0 소스 코드 분석과 macOS 실행 테스트를 바탕으로 작성했습니다(2026-05-23). /subgoal 기능은 실행 테스트를 하지 않았습니다.*

- 기본 사용법
- 작동 구조
- 턴 예산과 비용 통제
  - 모델 선택 가이드 (2026년 5월 기준)
- 안전장치
- 목표를 잘 쓰는 법
  - 검증 가능한 완료 조건을 넣는다
  - 큰 작업은 쪼갠다
  - 판정 모델이 정확도를 좌우한다
  - /goal에 맞지 않는 작업
- 세션 영속
- 게이트웨이에서의 작동
- 크론·서브에이전트와의 비교

크론이 "정해진 시각에 반복"하는 자동화라면, `/goal`은 **"목표를 달성할 때까지 연속"**하는 자동화입니다. 한 턴에 끝나지 않는 작업 — 수백 개 파일의 린트 수정, 대규모 리팩터링, 문서 전체 교정 — 을 목표로 설정하면, 에이전트가 매 턴 끝에 스스로 "다음 단계"를 이어갑니다.

기본 사용법

```
> /goal src/ 디렉터리의 모든 린트 에러를 수정하고 ruff check가 통과하도록 해줘
```

목표를 설정하면 그 텍스트가 곧바로 첫 턴의 사용자 메시지로 들어가고, 이후 매 턴 완료마다 판정 에이전트가 "목표가 달성됐는지"를 확인합니다. 달성되지 않았으면 연속 프롬프트가 자동 주입되어 다음 턴이 시작됩니다. 사용자가 직접 Enter를 누를 필요가 없습니다.

서브 명령:

명령	설명
`/goal <텍스트>`	목표 설정 — 첫 턴이 즉시 시작
`/goal` 또는 `/goal status`	현재 상태 표시 (☉ active , ⏸ paused , ✓ done)
`/goal pause`	일시정지 — 연속 턴 중단
`/goal resume`	재개 — 턴 예산을 초기화하고 이어서 진행
`/goal clear`	목표 해제 (`stop` , `done` 도 같은 작동)

작동 구조

```
flowchart TD
    A["사용자: /goal &amp;quot;린트 에러 전부 수정&amp;quot;"] ⟶ B["에이전트 턴"]
    B -- "응답 완료" --> C{"판정 에이전트"}
    C -- "done: false\n(연속 프롬프트 주입)" ⟶ B
    C -- "done: true" ⟶ D["목표 달성 ✓"]
```

판정 에이전트는 에이전트의 마지막 응답(최대 4,000자)과 원래 목표 텍스트를 받아, JSON(`{"done": true/false, "reason": "..."}`)으로 판정을 내립니다. 이 판정은 시스템 프롬프트를 건드리지 않으므로 프롬프트 캐시가 유지됩니다.

> Ralph loop
>
> 이 기능은 소스 코드에서 "Ralph loop"이라고 불립니다. OpenAI Codex CLI(v0.128.0)의 동명 기능에서 영감을 받아 Hermes에 도입되었습니다.

턴 예산과 비용 통제

기본 턴 예산은 **20턴**입니다. 예산이 소진되면 목표가 자동으로 일시정지되고, `/goal resume` 으로 재개하면 예산이 초기화됩니다.

```
# config.yaml
goals:
  max_turns: 20     # 기본값, 필요에 따라 조절
```

판정 에이전트는 보조 모델 슬롯 `auxiliary.goal_judge`를 사용합니다. 별도로 설정하지 않으면 메인 모델이 쓰이는데, 20턴짜리 루프에서 매 턴마다 메인 모델로 판정하면 비용이 누적됩니다. 저렴하고 빠른 모델을 지정하는 것이 좋습니다.

```
# config.yaml
auxiliary:
  goal_judge:
    provider: openrouter
    model: google/gemini-3-flash-preview
```

CLI로도 설정할 수 있습니다:

```
hermes config set auxiliary.goal_judge.provider openrouter
hermes config set auxiliary.goal_judge.model google/gemini-3-flash-preview
```

판정 LLM의 응답은 최대 4,096토큰(`auxiliary.goal_judge.max_tokens`로 조정 가능), temperature 0(결정적 판정)으로 제한됩니다.

모델 선택 가이드 (2026년 5월 기준)

`/goal`은 여러 턴에 걸쳐 실행되므로, 주 모델과 판정 모델의 조합이 비용과 성공률에 직접 영향을 줍니다.

주 모델 선택 기준 — `/goal` 루프에서 주 모델은 매 턴마다 도구를 호출해 작업을 수행합니다. 도구 호출 정확도가 높고, 긴 컨텍스트를 안정적으로 처리하는 모델이 턴 낭비를 줄입니다.

모델	비용 (입력/출력, 1M토큰)	특징
claude-sonnet-4.6	\$3.00 / \$15.00	도구 호출 안정성 최상[1]. 비용이 높지만 복잡한 작업에 확실
kimi-k2.6	\$0.73 / \$3.49	도구 호출 벤치마크 상위. 비용 대비 에이전트 성능 우수
minimax-m2.7	\$0.30 / \$1.20	Nous Research와 제품 파트너십[2]. 도구 호출 안정성도 상위[1]
deepseek-v4-pro	\$0.43 / \$0.87	극히 저렴하면서 에이전트 성능 상위. 텍스트 기반 자동화에 적합

판정 모델 선택 기준 — 판정 에이전트는 `{"done": true/false, "reason": "..."}` 형태의 짧은 JSON만 반환하면 됩니다. 구조화 출력 파싱이 정확하고 빠른 모델이면 충분합니다.

모델	비용 (입력/출력, 1M토큰)	특징
gemini-3-flash-preview	\$0.50 / \$3.00	구조화 출력에 강함. Hermes 소스 코드에서도 예시로 사용
step-3.5-flash	\$0.09 / \$0.30	극저가. 단순한 완료 조건 판정에 충분

> 💡 ***가격 시점**: OpenRouter 기준 가격이며, 모델 가격과 성능은 수시로 변동되므로 실행 시점에 다시 확인하는 것을 권장합니다. 로컬 모델(Ollama)을 판정 모델로 사용해 비용을 완전히 없앨 수도 있습니다(부록 「로컬 모델을 판정 모델로 활용하기」 참고).*

안전장치

Ctrl+C — 즉시 일시정지

에이전트가 턴을 실행하는 도중 Ctrl+C를 누르면, 판정을 거치지 않고 목표가 즉시 일시정지됩니다. 이 장치가 없으면 중단된 턴의 불완전한 응답이 "continue" 판정을 받아 바로 다음 턴이 시작되는 문제가 생깁니다.

빈 응답 건너뛰기

API 오류 등으로 에이전트 응답이 비어 있으면 판정 자체를 건너뜁니다. 불필요한 연속 턴이 시작되지 않습니다.

판정 파싱 실패 자동정지

판정 모델이 JSON이 아닌 응답을 3회 연속 반환하면, 목표가 자동 일시정지되고 `auxiliary.goal_judge` 설정 안내가 출력됩니다. 메인 모델이 아닌 저렴한 모델을 판정에 쓸 때 발생할 수 있으므로, 이 메시지가 보이면 판정 모델을 바꿔 보는 것이 좋습니다.

사용자 입력 우선

목표가 활성 상태에서도 사용자가 메시지를 입력하면 연속 턴보다 사용자 입력이 우선 처리됩니다. 사용자 턴이 끝난 후 다시 목표 루프로 돌아갑니다.

목표를 잘 쓰는 법

판정 에이전트는 **원래 목표 텍스트**와 **에이전트의 마지막 응답(최대 4,000자)**만 보고 완료 여부를 판단합니다. 목표를 어떻게 쓰느냐에 따라 결과 품질뿐 아니라 소요 시간과 토큰 비용이 크게 달라집니다.

검증 가능한 완료 조건을 넣는다

판정 에이전트가 "끝났다"를 확신하려면, 에이전트의 마지막 응답에서 완료를 객관적으로 확인할 수 있어야 합니다.

모호한 목표	구체적인 목표
코드를 깨끗하게 정리해줘	`src/` 디렉터리에서 `ruff check`가 경고 0개로 통과하도록 수정해줘
블로그 글 몇 개 써줘	"AI 에이전트" 주제로 블로그 5편을 작성하고, 각각 `posts/` 폴더에 마크다운으로 저장해줘
테스트를 보강해줘	`tests/` 폴더의 커버리지를 80% 이상으로 올려줘. `pytest --cov`로 확인

왼쪽 같은 목표는 에이전트가 일부만 고쳐도 판정이 "done"을 내릴 수 있고, 반대로 다 끝났는데도 "done"이 안 나와 턴 예산이 소진될 수 있습니다.

큰 작업은 쪼갠다

수백 개 파일을 고치는 작업을 하나의 `/goal`로 설정하면, 20턴 예산 안에 끝나지 않거나 중간에 방향이 틀어질 위험이 있습니다. 범위를 좁혀서 여러 번 나누는 것이 안전합니다.

```
# 한 번에 전부 — 위험
/goal 프로젝트 전체의 경어체를 통일하고 린트 에러를 수정해줘

# 챕터 단위로 나눠서 — 안전
/goal pages/01-*.md 파일의 경어체를 "~합니다/~입니다"로 통일해줘
```

한 묶음이 끝나면 결과를 확인하고, 다음 `/goal`을 설정합니다. 목표 큐잉(여러 목표를 미리 대기시키는 기능)은 아직 지원되지 않으므로, 이 과정은 수동입니다.

판정 모델이 정확도를 좌우한다

`auxiliary.goal_judge`에 저렴한 모델을 지정하면 비용은 줄지만, 복잡한 목표에서 조기 완료 판정(실제로 끝나지 않았는데 "done")이 늘어납니다. 목표가 단순하면 Flash급으로 충분하고, 판단이 까다로운 작업이면 Sonnet급 이상을 쓰는 것이 턴 낭비를 줄입니다.

`/goal`에 맞지 않는 작업

"문장을 자연스럽게 다듬어줘", "코드 구조를 개선해줘"처럼 **완료 기준이 주관적인 작업**은 `/goal`과 맞지 않습니다. 판정 에이전트도 LLM이라서, 사람도 의견이 갈리는 판단을 안정적으로 내리기 어렵습니다. 이런 작업에는 사용자가 턴마다 결과를 확인하면서 진행하는 일반 대화가 더 적합합니다.

세션 영속

목표 상태는 세션 DB에 저장됩니다. CLI를 닫았다가 다음 날 `/resume`으로 돌아와도 목표가 이어집니다. `clear`로 해제한 목표도 DB에서 삭제되지 않고 `status="cleared"`로 남아 있어, 어떤 목표를 설정했었는지 나중에 확인할 수 있습니다.

게이트웨이에서의 작동

CLI, 게이트웨이(Telegram·Discord 등), TUI 세 곳 모두 같은 `GoalManager`를 사용합니다. 게이트웨이에서는 연속 프롬프트가 메시지 큐에 들어가므로, 사용자 메시지가 자연스럽게 우선합니다. 에이전트 턴이 실행 중일 때는 새 목표 텍스트 설정이 거부되고, `status` · `pause` · `clear`만 허용됩니다.

크론·서브에이전트와의 비교

	크론	`/goal`	`delegate_task`
트리거	시각 기반 (매일, 매시간)	목표 텍스트	메인 에이전트의 판단
실행 패턴	독립 세션, 반복	같은 세션, 연속 턴	별도 세션, 1회
세션 간 컨텍스트	없음 (매 실행 격리)	유지 (같은 대화 이력)	없음
종료 조건	삭제할 때까지 반복	판정 에이전트의 "done"	작업 완료 시
적합한 작업	정기 점검, 리포트	대규모 수정, 단계적 작업	단순 병렬 위임

`/goal`이 크론과 다른 핵심은 **대화 컨텍스트가 유지된다**는 점입니다. 이전 턴에서 무엇을 고쳤는지 에이전트가 기억하고 있으므로, "이미 고친 파일을 또 고치는" 낭비가 줄어듭니다.

---

1. Remote OpenClaw, "Best Chinese Models for Hermes Agent" (2026-05-22). DeepSeek V4와 MiniMax M2.7이 Claude Sonnet 4.6의 도구 호출 일관성에 가장 근접하다고 평가. ↩↩
2. Nous Research(@NousResearch), X 포스트. MiniMax와 제품·모델 양면에서 파트너십을 맺고 있으며, MaxHermes(관리형 클라우드)를 공동 운영 중. ↩